HKDSE ICT SBA Gomoku Project Technical Document

Author: Jimmy Yang
Project: Terminal-based Gomoku (Five-in-a-Row) Implementation

---

Section 1: Introduction

This technical document outlines the development and implementation of a Gomoku (Five-in-a-Row) game created for my School-Based Assessment (SBA). The program is written in **Python 3.14.3** and is designed for cross-platform execution. It is fully compatible with **Windows Terminal** (NT environments) and **POSIX-compliant terminals** (macOS and Linux) utilizing a Python (>=3.14.3) interpreter.

The primary objective of this project was to develop a stable, text-based version of the 15×15 standard Gomoku game. The program facilitates a two-player experience via a coordinate-based input system. Key technical features include:

1. **Dynamic Rendering:** Automatic board refreshing and screen clearing for a clean UI.
2. **Algorithmic Win-Detection:** A localized search algorithm to identify a winner immediately upon a valid move.
3. **Error Handling:** Robust validation of user input to prevent program crashes.

The development was conducted on **Arch Linux x86_64**, adhering to a "zero-dependency" philosophy by relying solely on the Python Standard Library to maximize portability.

Section 2: Data & Variables

A crucial part of the development process was the selection of data types and data structures to represent the game state efficiently.

2.1 Data Structures

Board Presentation: A **two-dimensional (2D) array (list of lists)** was chosen to represent the 15×15 chessboard.

- **Justification:** The 2D array maps directly to the Cartesian coordinate system of a physical board. This allows for high-speed data access and manipulation using the

index format `board[row][col]`.

The following code snippet demonstrates the efficiency of accessing this structure during the winner recognition process:

```
while 0 <= nr < SIZE and 0 <= nc < SIZE and board[nr][nc] == p:
```

2.2 Global State and Constants

To maintain a consistent game state, several key variables and constants were defined:

- **Constants:** `SIZE`, `EMPTY`, `BLACK`, and `WHITE` are defined at the top of the script. These act as global settings that make the code easier to modify (e.g., changing the board size or character symbols).
- **Status Variables:**
  - `current_player`: Stores the state of the active turn. This is vital for the logic to distinguish between 'X' and 'O' during the win-check phase.
  - `status_msg`: This acts as a **temporary message stack**. Because the terminal screen is cleared at the start of every frame to update the board, we need a persistent variable to "carry over" messages (like errors or move confirmations) so they can be displayed on the refreshed board.

Section 3: Module Composition

The program follows the principle of **Modular Programming**, breaking down complex tasks into smaller, reusable functions.

Section 3 Part 1: Parsing Input (Data Validation)

This module is responsible for **String Manipulation** and **Data Validation**. It translates user-friendly input (e.g., "A8") into mathematical offsets (indices) that the program can process.

```
def parse_input(input_str):
    s = input_str.strip().upper()
    if s == 'Q':
        return 'QUIT', ""
```

First, the input is sanitized using `.strip()` and `.upper()` to handle accidental spaces or lowercase letters, improving the **User Experience (UX)**.

```
if len(s) < 2:
    return None, "Coordinate too short."
```

Next, a length check ensures the input is at least two characters long (one letter, one number), preventing **IndexErrors** later in the code.

```
try:
    col = ord(s[0]) - ord('A')
    row = int(s[1:]) - 1

    if 0 <= col < SIZE and 0 <= row < SIZE:
        return (row, col), ""
    else:
        return None, "Coordinate out of range."
except (ValueError, IndexError):
    return None, "Invalid input."
```

Algorithm Analysis: The module uses **Exception Handling** (`try...except`) rather than nested `if-else` statements. This is known as the **EAFP** (Easier to Ask for Forgiveness than Permission) approach in Python.

- **Benefit 1:** It separates the "happy path" (correct input) from "error paths," making the code more readable.
- **Benefit 2:** It catches runtime errors—such as a user typing "A!" (which would crash an `int()` conversion)—and provides a graceful error message instead of a crash.
- **Benefit 3:** It provides feedback to the user via the `status_msg` stack if the coordinate is "Out of range."

Section 3 Part 2: Clearing Screen (Interface Management)

To avoid a "scrolling terminal" effect, the `clear_screen` function manages the terminal display.

```
def clear_screen():
    os.system('cls' if os.name == 'nt' else 'clear')
```

This function demonstrates **platform-independent logic**. It detects the operating system using `os.name`. If the system is `nt` (Windows), it sends the `cls` command. For all other systems (UNIX, Linux, macOS), it sends the `clear` command. This ensures a professional UI regardless of the user's OS.

Section 3 Part 3: Creating Board (Initialization)

```
def create_board():
    return [[EMPTY for _ in range(SIZE)] for _ in range(SIZE)]
```

This function uses **List Comprehension** to initialize the 2D array. This is a memory-efficient way to populate the grid with `EMPTY` characters at the start of every game.

Section 3 Part 4: Printing Board (Output Rendering)

This part of the program handles the visual output. The board is rendered row-by-row with dynamic coordinate headers.

```
def print_board(board, status_msg, current_player):
    clear_screen()
```

The board begins by clearing the previous frame. Then, it generates the A-O headers.

```
    letters = [chr(ord('A') + i) for i in range(SIZE)]
    print("\n     " + " ".join(letters))
    print("    +" + "--" * SIZE + "+")
```

For the rows, a `for` loop is used in conjunction with `.rjust(2)`.

- **Justification:** Using `.rjust(2)` ensures that row numbers 1 through 15 are properly aligned. Without this, single-digit numbers would cause the board border to shift, damaging the visual integrity of the grid.

```
    for i in range(SIZE):
        row_num = str(i + 1).rjust(2)
        row_content = " ".join(board[i])
        print(f" {row_num} | {row_content} | {row_num}")
```

Finally, the metadata (Current Player and System Messages) are printed at the bottom. By bundling these into the `print_board` function, we ensure that the user always sees their current status and any errors immediately below the board.

Section 3 Part 5: Winner Checking (Algorithm Design)

Winner recognition is the most critical and complex algorithm in this script.

Brute Force vs. Localized Search: A "brute force" scan would check the entire board for five-in-a-row every turn. This is computationally expensive ($O(N^2)$). Instead, I developed a **localized search algorithm**. It only checks the four axes (horizontal, vertical, and two diagonals) intersecting the *last stone placed*.

```
def check_win(board, rowc, column, player):
  directions = [(0, 1), (1, 0), (1, 1), (1, -1)]
```

The algorithm iterates through these four direction tuples. For each direction, it performs a **Bidirectional Search**:

1. **Forward Traversal:** It steps forward from the current point until it hits an edge or a different colored stone, incrementing the `count`.
2. **Backward Traversal:** It steps in the opposite direction from the same starting point to complete the check.

```
for change_r, change_c in directions:
  count = 1
  # Forward search
  new_r, new_c = rowc + change_r, column + change_c
  while 0 <= new_r < SIZE and 0 <= new_c < SIZE and \
    board[new_r][new_c] == player:
    count += 1
    new_r += change_r
    new_c += change_c
  # Backward search
  # ... logic repeated for opposite direction
```

If the final `count` is ≥ 5, the function returns `True`. This algorithm is highly efficient and provides instantaneous feedback even on low-powered hardware.

Section 4: Main Loop (Control Logic)

The `main()` function serves as the **Program Controller**, orchestrating the flow between the board, the player, and the logic modules.

At the start, it initializes the data structures and sets the starting player:

```
def main():
    board = create_board()
    current_player = BLACK
    status_msg = f"Player {current_player} please place."
```

The `while True` loop keeps the game running indefinitely until a **Termination Condition** is met:

1. **Interrupt Signal:** If the user enters "Q", the `break` command exits the loop.
2. **Win Condition:** If `check_win` returns `True`, the game displays the final board and ends.
3. **Draw Condition:** Using the `all()` function, the script checks if every cell is filled. If no winner is found and the board is full, it declares a draw.

```
    if all(cell != EMPTY for row in board for cell in row):
        print_board(board, "It is a draw!", current_player)
        break
```

State Switching Logic: At the end of a successful turn, the script uses a **ternary-style assignment** to toggle the players:

```
current_player = WHITE if current_player == BLACK else BLACK
```

Finally, the `if __name__ == "__main__":` block is used to ensure the program only runs when executed directly. This is a best practice for **module reusability** and **portability**.

Section 5: Future Development Plan

To further enhance the project, I have identified two primary areas for expansion:

5.1 Artificial Intelligence Implementation (PvE)

Adding a "Player vs Environment" mode is a high priority. I plan to explore two AI methodologies:

1. **Symbolism AI (Minimax Algorithm):** A traditional mathematical model that uses tree-searching with depth-first search (DFS) to predict optimal moves.
2. **Connectionism AI (CNN):** Researching models like **Rapfi**, which utilizes Convolutional Neural Networks (CNN) to recognize patterns in the board state. This represents the cutting edge of Gomoku AI.

5.2 UI/UX Improvements

While the terminal interface is functional and highly portable, adding a graphical welcome screen or utilizing the `pygame` library could provide a more modern user experience. However, the current terminal-based design is intentionally maintained to ensure the game remains lightweight and runs without external library dependencies.

Section 6: Reusability and Portability

This script is highly portable due to the following design choices:

- **Cross-Platform Coding:** Tested on **Arch Linux (6.19.9 kernel)** using the Kitty terminal and **Windows 11 (25H2)** via Windows Terminal.
- **Modular Architecture:** The script can be imported as a module into a larger software suite without modification.
- **Compatibility:** It supports **musl C library** (Chimera Linux) and **aarch64** (ARM) architectures, ensuring it runs on everything from desktop PCs to Raspberry Pi devices.

Section 7: References

1. Python 3.14.3 Documentation: https://docs.python.org/3/
2. Microsoft Learn - Windows Terminal Customize: https://learn.microsoft.com/
3. Rapfi Open Source Project (Gomoku AI): https://github.com/dhbloo/rapfi
4. Markdown Syntax Standard: https://markdown.com.cn/

Section 8: Redistribution and Availability

This project is released under the **MIT License**, permitting free use and redistribution.

The project source and documentation are hosted at:
https://github.com/sarosaka-orz/HKDSE-ICT-SBA-Gomoku